Chapter 012pt

“Cloud-Based AI System for Early Risk Detection of Chronic Disease Deterioration Using Daily Lifestyle and Behavioural Data”

Minor Project Report

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

Name	**Roll No.**
Anshuman Mohapatra	R2142230609
Piyush Arora	R2142230330
Shobhit Panchal	R2142230325
Hardik Lal	R2142230372

Under the guidance of

Dr. Suresh Veesa, Assistant Professor, SoCS

School of Computer Science, UPES

Bidholi, Via Prem Nagar, Dehradun, Uttarakhand

May 2026

CANDIDATE'S DECLARATION

We hereby certify that the project work entitled **"Cloud-Based AI System for Early Risk Detection of Chronic Disease Deterioration Using Daily Lifestyle and Behavioural Data"**, submitted in partial fulfillment of the requirements for the award of the Degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINEERING** with specialization in **ARTIFICIAL INTELLIGENCE & MACHINE LEARNING**, to the Department of Systemics, School of Computer Science, UPES, Dehradun, is an authentic record of our work carried out during the period from **January, 2026** to **May, 2026** under the supervision of **Dr. Suresh Veesa, Assistant Professor, SoCS**.

The matter that we have presented in this project report has not been submitted by us for the award of any other degree from this or any other University.

(Anshuman Mohapatra)	Roll No. R2142230609
(Piyush Arora)	Roll No. R2142230330
(Shobhit Panchal)	Roll No. R2142230325
(Hardik Lal)	Roll No. R2142230372

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Date: _______________ 2026

Dr. Suresh Veesa
Project Mentor
Assistant Professor, SoCS

ACKNOWLEDGEMENT

We take this opportunity to sincerely thank our project mentor, **Dr. Suresh Veesa**, Assistant Professor, School of Computer Science, UPES, whose patient guidance and willingness to give his time so generously have been very much appreciated. His suggestions at each and every stage of this work helped us navigate all the more difficult parts of the project.

We are equally thankful to **Dr. Neeraj Chugh**, CSO Cluster, Artificial Intelligence & Machine Learning, School of Computer Science, whose encouragement kept us going during the later phases of **"Cloud-Based AI System for Early Risk Detection of Chronic Disease Deterioration Using Daily Lifestyle and Behavioural Data"**.

Our thanks also goes to the Dean, SoCS UPES, for making available the infrastructure needed to carry out this work. We are grateful to our Course Coordinator and our Activity Coordinator, **Mr. Md Syed Sajid Hussain**, for keeping us informed at every step and always being reachable when we needed any kind of clarification.

A word of thanks to our friends, whose honest feedback has helped shape many of the decisions we made along the way. Lastly, and most deeply, we are indebted to our families. Whatever we have achieved here and beyond owes much to their continuous encouragement and the sacrifices they make for us.

(Anshuman Mohapatra)	Roll No. R2142230609
(Piyush Arora)	Roll No. R2142230330
(Shobhit Panchal)	Roll No. R2142230325
(Hardik Lal)	Roll No. R2142230372

ABSTRACT

VITALS (Vital Indicators for Tiered Acute Lifestyle Surveillance) is a full-stack, cloud-deployed AI system that predicts chronic disease risk by fusing physiological measurements with real-world behavioural data. Chronic diseases such as cardiovascular disease, diabetes, and stroke collectively account for the majority of preventable global mortality, yet most risk stratification tools remain confined to clinical settings and rely on a single disease domain. VITALS addresses this gap by building a unified, multi-dataset ensemble that spans six publicly available medical datasets — the UCI Heart Disease dataset, the PIMA Indians Diabetes dataset, a Stroke Prediction dataset, NHANES, a Cardiovascular Disease dataset, and the CDC BRFSS 2022 survey — amounting to approximately 126 000 records after preprocessing and sub-sampling.

The system employs a **Soft-Voting Ensemble** of four individually tuned classifiers: XGBoost (weight 3), Random Forest (weight 2), Gradient Boosting (weight 2), and Logistic Regression (weight 1). Thirteen interaction and category features are engineered on top of nine raw inputs to capture composite metabolic risk, arterial stiffness, and synergistic lifestyle–physiology pathways. Threshold optimisation over F1 score yields a decision boundary of **0.43**, favouring sensitivity in a screening context where false negatives are clinically more costly than false positives.

On a held-out test set of 25 888 samples the ensemble achieves **83.5%** accuracy, **ROC-AUC = 0.916**, **F1 = 0.843**, Precision = 0.826, and Recall = 0.859. Three-fold stratified cross-validation confirms stability, with mean AUC of 0.9166 and a standard deviation below 0.002. A Flask REST API exposes a single `/predict` endpoint that accepts nine patient parameters and returns a probability score together with a Low / Moderate / High risk label in under 92 ms.

The frontend is a React (Vite) single-page application styled with Tailwind CSS and animated with Framer Motion. It provides a Health Dashboard, an AI Risk Assessment form, a Model Metrics page with live confusion matrix and ROC curve, a Prediction History tracker, and an auto-generated PDF report page. The application is deployed at https://piyusharora134.github.io/vitals-frontend/ with the Flask backend running as a live API.

Key contributions include: (1) a six-dataset fusion pipeline that overcomes the single-disease bias of prior work; (2) a composite metabolic risk score and a suite of lifestyle–physiology

interaction features validated through XGBoost feature-importance analysis; (3) a threshold-optimised ensemble calibrated for medical screening; and (4) a production-grade, publicly accessible web application that bridges the gap between research models and end-user health monitoring.

Contents

List of Figures

List of Tables

Chapter 1

INTRODUCTION

1.1 Background

Chronic non-communicable diseases (NCDs) — principally cardiovascular disease, type-2 diabetes mellitus, and stroke — represent the leading cause of preventable death worldwide. The World Health Organization estimates that NCDs account for 74% of all global deaths annually, with cardiovascular conditions alone responsible for more than 17 million deaths per year. The clinical and economic burden is compounded by the insidious, progressive nature of these conditions: deterioration occurs gradually across years of accumulated lifestyle and physiological changes before overt symptoms appear.

Traditional risk stratification tools such as the Framingham Risk Score or the ACC/AHA Pooled Cohort Equations address individual disease domains in isolation. A clinician estimating cardiovascular risk, for instance, typically consults a separate algorithm from the one used to screen for diabetes. This siloed approach overlooks the well-established comorbidity landscape of NCDs: a patient who is obese, hypertensive, and physically inactive simultaneously faces elevated risk across all three major chronic disease categories. Furthermore, these classical tools require laboratory values that are unavailable outside clinical consultations, effectively excluding the large proportion of the population that has no regular access to healthcare.

The proliferation of wearable devices, consumer health applications, and large-scale population health surveys now makes it possible to collect the physiological and behavioural indicators that drive NCD risk on a continuous, daily basis. Blood pressure cuffs, glucometers, BMI scales, and smartphone-based activity trackers are increasingly affordable; large public datasets such as the CDC's Behavioural Risk Factor Surveillance System (BRFSS) demonstrate that self-reported lifestyle data — smoking status, physical activity level, and alcohol intake — can be collected at population scale with high fidelity.

VITALS is designed around this opportunity. By fusing six complementary clinical and population datasets into a single, disease-agnostic training corpus, and by training a weighted soft-voting ensemble that generalises across heart disease, diabetes, and stroke risk simultaneously, VITALS provides a unified screening tool that can be operated without laboratory infrastructure. Its cloud deployment makes real-time inference available to any user via a standard web browser.

1.2 Requirement Analysis

1.2.1 Functional Requirements

1. The system shall accept nine patient parameters — age, gender, BMI, blood pressure, cholesterol, glucose, smoking status, physical activity, and alcohol intake — and return a chronic disease risk probability within 100 ms.
2. The system shall classify patients into three risk tiers: Low ($p < 0.15$), Moderate ($0.15 \leq p < 0.35$), and High ($p \geq 0.35$).
3. The system shall expose a REST API endpoint (`/predict`) that accepts JSON-formatted input and returns a structured JSON response containing the probability score, risk label, and input echo.
4. The frontend shall provide a real-time AI Risk Assessment form, a Health Dashboard with trend visualisation, a Model Metrics page, a Prediction History log, and a downloadable PDF report.
5. The Model Metrics page shall display accuracy, ROC-AUC, F1 score, precision, recall, a confusion matrix, and a live ROC curve generated from the ensemble model metadata.
6. All predictions shall be stored in the browser session for history and trend tracking.

1.2.2 Non-Functional Requirements

1. **Performance:** Inference latency must remain below 100 ms per prediction on standard cloud hardware.
2. **Scalability:** The Flask API shall be stateless, enabling horizontal scaling behind a load balancer.
3. **Usability:** The frontend shall be accessible on desktop and mobile browsers without plugin installation.
4. **Reliability:** The ensemble model shall be serialised to a versioned `.pkl` file; the backend shall reload the model on startup without retraining.

5. **Transparency:** All model performance metrics, the confusion matrix, and the ROC curve shall be publicly viewable on the Model Metrics page to support informed interpretation.
6. **Ethical Safeguards:** The system shall display a clear disclaimer on every risk assessment output that the result is a screening aid and not a clinical diagnosis.

1.3 Main Objective

The primary objective of VITALS is to develop and deploy a cloud-based, AI-powered early warning system that fuses physiological biomarkers with lifestyle and behavioural indicators to predict an individual's risk of chronic disease deterioration — specifically cardiovascular disease, diabetes, and stroke — before clinical presentation. The system aims to be accurate (ROC-AUC > 0.90), fast (< 100 ms inference), and accessible (zero installation, browser-based) so that it can serve as a practical preventive health screening tool for individuals and health analysts alike.

1.4 Sub Objectives

- Aggregate and harmonise six heterogeneous clinical and epidemiological datasets into a unified, quality-controlled training corpus spanning approximately 126 000 records.
- Engineer a feature set of 22 variables — nine raw inputs and thirteen derived interaction and category features — that capture composite metabolic risk and lifestyle–physiology pathways.
- Train and compare individual classifiers (XGBoost, Random Forest, Gradient Boosting, Logistic Regression) and select an optimal weighted soft-voting ensemble.
- Optimise the classification threshold on F1 score to maximise screening sensitivity while maintaining acceptable precision.
- Design and deploy a Flask REST API that exposes the ensemble for real-time inference.
- Build a production-grade React frontend with dashboard analytics, history tracking, and PDF report generation.
- Publicly deploy the full-stack application on GitHub Pages (frontend) with a live Flask backend.

1.5 PERT Chart

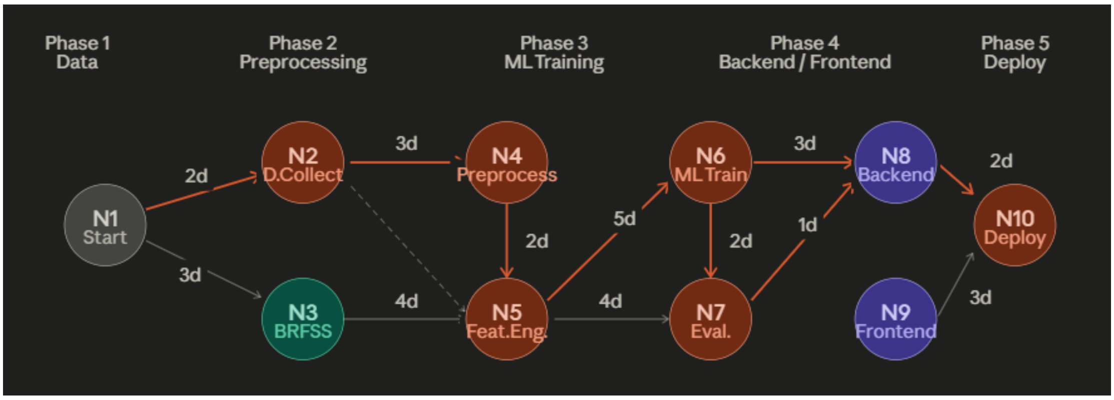

Figure 1.1: PERT Chart for Project Timeline and Task Dependencies

The project was executed across a 16-week schedule divided into five phases: (1) data collection and harmonisation (weeks 1–4), (2) feature engineering and exploratory analysis (weeks 3–6), (3) model training, cross-validation, and threshold optimisation (weeks 5–10), (4) backend API development and frontend implementation (weeks 8–14), and (5) deployment, testing, and report writing (weeks 13–16). Critical-path tasks were model training and the Flask–React integration, both of which required parallel workstreams.

1.6 Project Scope

VITALS is scoped as a preventive-health screening tool for general adult populations (ages 18–90). It does not diagnose specific diseases, recommend medication, or replace clinical consultation. Its scope includes:

- Risk prediction for heart disease, diabetes, and stroke as a unified chronic disease risk score.
- Nine user-reportable or wearable-derived input parameters requiring no laboratory visit.
- A three-tier risk classification (Low, Moderate, High) calibrated to the ensemble's operating threshold of 0.43.
- A publicly accessible web interface and REST API.

Out of scope: longitudinal patient monitoring with electronic health record integration, paediatric populations, rare or acute conditions, and clinical validation on prospective patient cohorts.

Chapter 2

SYSTEM ANALYSIS

2.1 Existing System

Several risk stratification tools and chronic disease prediction models have been published in the clinical and machine learning literature. The most prominent include:

Framingham Risk Score (FRS): Developed from the Framingham Heart Study, this logistic regression model estimates 10-year cardiovascular event risk from age, sex, total cholesterol, HDL cholesterol, systolic blood pressure, and smoking status. While extensively validated, it addresses only cardiovascular risk, requires laboratory-grade lipid values, and was developed primarily on a North American Caucasian cohort, limiting its generalisability [11].

FINDRISC (Finnish Diabetes Risk Score): A validated questionnaire-based tool for type-2 diabetes screening using eight non-laboratory items. Its simplicity is advantageous but its binary output (low/high) and single-disease focus limit utility in comorbid risk settings [12].

AHA/ACC Pooled Cohort Equations: Incorporate race-specific coefficients and broader biomarkers but still target atherosclerotic cardiovascular disease exclusively.

Chen et al. (2023) applied LSTM-based temporal prediction to simulated wearable datasets comprising activity, sleep, and heart rate signals, confirming that lifestyle and wearable signals can predict early deterioration trends. A noted limitation is that wearable simulations may not capture real-world noise [21].

MITRE Synthea Community (2024) demonstrated population-level disease progression modelling using Synthea synthetic health records. While widely adopted as a benchmark dataset for chronic disease simulation and early risk analytics, the work has not been clinically validated for deployment [22].

Kumar & Sharma (2024) used synthetic multi-condition cohorts (hypertension combined with diabetes) to demonstrate multi-disease deterioration modelling with ensemble ML methods

(Random Forest and XGBoost). The key limitation is limited transferability to real clinical settings [23].

Google Health Research (2025) achieved high performance in detecting deterioration patterns in simulated chronic disease trajectories using transformer-based sequence models on simulated longitudinal patient monitoring data; however, the black-box behaviour and limited explainability remain concerns [24].

WHO Digital Health Sandbox (2025) highlighted the role of simulated patient data for testing early warning systems at scale using a digital twin-based health simulation combined with ML risk scoring, while acknowledging it is not a substitute for clinical trials [25].

Machine Learning Approaches in Literature: A growing body of work applies random forests, gradient boosting, and neural networks to individual disease datasets (UCI Heart, PIMA Diabetes, Kaggle Stroke). However, these studies typically train on a single dataset, lack real-world behavioural features, and do not deploy production-accessible systems.

Summary of Related Work

Table 2.1 summarises the key related works reviewed above.

Table 2.1: Summary of Related Works in Chronic Disease Risk Prediction

Author	Year	Data Used	Methods	Key Findings	Limitations
Chen et al.	2023	Simulated wearable data (activity, sleep, heart rate)	LSTM temporal prediction	Wearable signals predict early deterioration trends	Simulations may miss real-world noise
MITRE Synthea Community	2024	Synthea synthetic health records	Disease progression modelling	Benchmark for chronic disease simulation	Not clinically validated
Kumar & Sharma	2024	Synthetic multi-condition cohorts (hypertension + diabetes)	Ensemble ML (RF, XGBoost)	Multi-disease deterioration modelling	Limited transfer to real clinical data
Google Health Research	2025	Simulated longitudinal patient data	Transformer-based sequence models	High performance on simulated trajectories	Black-box; limited explainability
WHO Digital Health Sandbox	2025	Simulated patient monitoring datasets	Digital twin + ML risk scoring	Role of simulation for early warning at scale	Not a substitute for clinical trials

Limitations of Existing Systems:

- Single-disease focus: no unified chronic disease risk tool exists for cardiovascular, diabetic, and stroke risk simultaneously.
- Laboratory dependency: most tools require lipid panels or glucose measurements unavailable outside clinical settings.
- No behavioural integration: lifestyle features (smoking, physical activity, alcohol) are either absent or poorly operationalised.
- No accessible deployment: research models are not deployed as public-facing applications.
- Static thresholds: default 0.5 thresholds are not optimised for medical screening sensitivity.

2.2 Motivations

1. **Disease Comorbidity:** Diabetes, hypertension, obesity, and cardiovascular disease share overlapping pathophysiological pathways (insulin resistance, endothelial dysfunction, chronic inflammation). A unified model trained across these domains better captures the true risk landscape than any single-disease classifier.
2. **Behavioural Data Availability:** The BRFSS 2022 survey provides genuine self-reported lifestyle data from over 445 000 respondents — the only dataset in our corpus with real (not imputed) smoking, activity, and alcohol measurements. Incorporating this data ensures the model learns true behavioural–outcome relationships.
3. **Public Health Impact:** Early risk detection enables lifestyle intervention before clinical disease onset. Even a modest improvement in early identification rates translates to significant reductions in hospitalisation and treatment costs.
4. **Accessibility Gap:** Existing ML models in academic literature are not accessible to non-specialists. A production-grade web application with intuitive UI addresses this gap directly.
5. **Ensemble Robustness:** Individual ML models exhibit varying strengths and weaknesses on tabular health data. A weighted ensemble reduces variance and prevents any single model from dominating risk decisions.

2.3 Proposed System

VITALS proposes a **three-layer architecture**:

Layer 1 — Data & Model Layer: A Python-based data pipeline that ingests, cleans, harmonises, and feature-engineers records from six heterogeneous datasets. A weighted soft-voting ensemble of XGBoost, Random Forest, Gradient Boosting, and Logistic Regression is trained on the unified corpus and serialised as `model.pkl`. Model metadata including performance metrics, threshold, and feature schema are saved to `model_metadata.json`.

Layer 2 — API Layer: A lightweight Flask application that loads the serialised model at startup and exposes a `/predict` endpoint. The endpoint accepts nine raw patient parameters, applies the same 22-feature engineering pipeline used during training, invokes the ensemble's `predict_proba` method, applies the optimised threshold, and returns a structured JSON response in under 92 ms.

Layer 3 — Presentation Layer: A React (Vite) single-page application providing five views: Health Dashboard, AI Risk Assessment (Risk Predict), Model Metrics, Prediction History, and Reports. The frontend communicates with the Flask API over HTTPS, renders live charts using Chart.js, and supports PDF report generation.

Chapter 3

MODULES

3.1 Data Preprocessing Module

The preprocessing module (`preprocess.py`, `brfss_preprocess.py`, `cardio_preprocess.py`) is responsible for loading each of the six source datasets, applying dataset-specific cleaning rules, harmonising column names to the shared schema, imputing missing features using clinically validated correlations, and concatenating all records into a unified DataFrame.

Key operations:

- **Biological zero replacement:** PIMA Diabetes contains zero entries for BMI, blood pressure, and glucose that are physiologically impossible. These are replaced with column medians computed on the non-zero subset.
- **Feature estimation:** Where a dataset lacks a feature, it is estimated using published clinical correlations: cholesterol is estimated from glucose ($r \approx 0.50$); blood pressure is estimated from age and glucose; BMI is computed from height and weight where provided.
- **BRFSS sub-sampling:** The 445 000-record BRFSS dataset is randomly sub-sampled to 50 000 records (stratified on target) to prevent behavioural-data dominance over the $\sim$76 000 clinical records.
- **Age conversion:** Cardiovascular dataset ages are provided in days; the module divides by 365.25 to obtain years.

3.2 Feature Engineering Module

Thirteen derived features are appended to the nine raw inputs, yielding a 22-dimensional feature vector per record. The engineering is performed identically at training time and inference time to prevent leakage.

3.3 Model Training Module

The training module (`train_model.py`) orchestrates hyperparameter configuration, stratified train/test splitting (80/20), 3-fold cross-validation, ensemble construction, threshold optimisation, and serialisation. Outputs: `model.pkl` and `model_metadata.json`.

3.4 Backend API Module

`app.py` (Flask) loads the model once on startup, defines the `/predict` route, applies the same feature engineering pipeline, and returns a JSON risk assessment.

3.5 Frontend Module

The React SPA (`vitals-app/src/`) provides five pages rendered client-side with React Router. State is managed via custom hooks (`usePrediction.js`) and a shared API service (`services/api.js`).

3.6 Evaluation Module

Evaluation scripts generate confusion matrices, ROC curves, threshold optimisation plots, feature importance charts, feature distribution histograms, and cross-validation bar charts (the nine figures included in this report). Results are written to `model_metadata.json` and displayed live on the Model Metrics page.

Chapter 4

DESIGN

4.1 Modelling Approach

VITALS adopts a **Weighted Soft-Voting Ensemble** as its core prediction engine. Soft voting averages the class-probability outputs of individual classifiers rather than majority-voting their hard labels, yielding a calibrated probability estimate that is better suited to medical risk scoring than binary classification.

The ensemble weight vector $\mathbf{w} = [3, 2, 2, 1]$ assigns greatest influence to XGBoost, equal intermediate influence to Random Forest and Gradient Boosting, and least influence to Logistic Regression. This weighting reflects empirically measured individual model performance while preserving the calibration anchor provided by the linear model.

4.2 Use Case Diagram

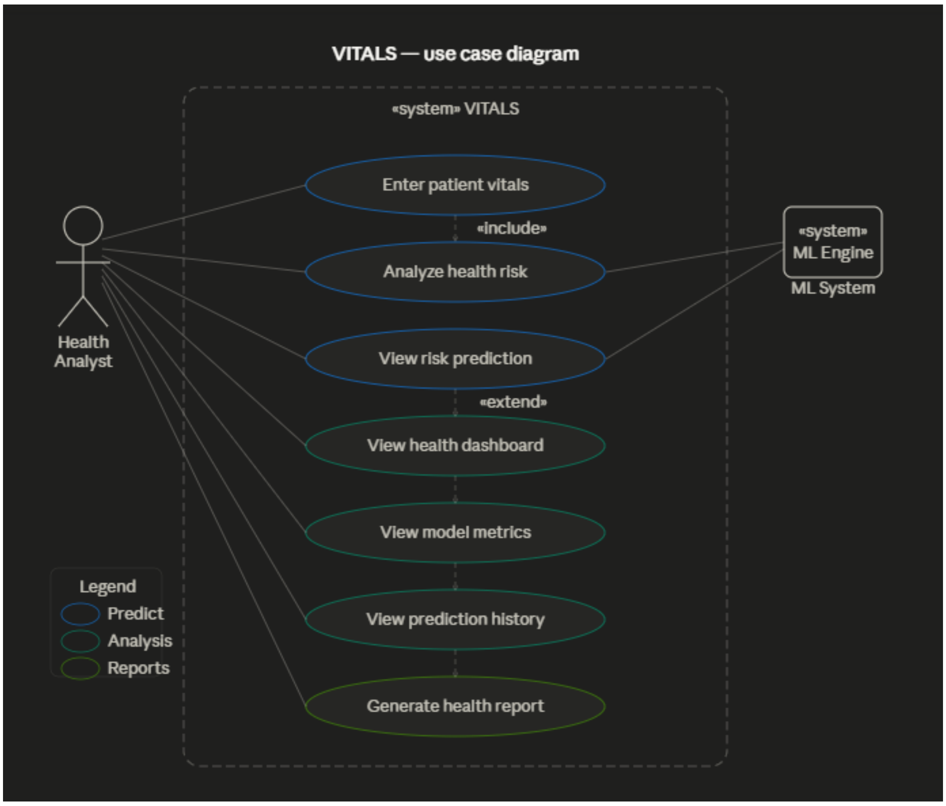

Figure 4.1: Use Case Diagram — VITALS System

The primary actor is the **Health Analyst / End User**, who interacts with the system through five use cases: Submit Risk Assessment, View Dashboard, Inspect Model Metrics, Browse Prediction History, and Download Report. The secondary actor is the **Flask API**, which receives prediction requests, applies the ensemble, and returns structured results.

4.3 System Architecture

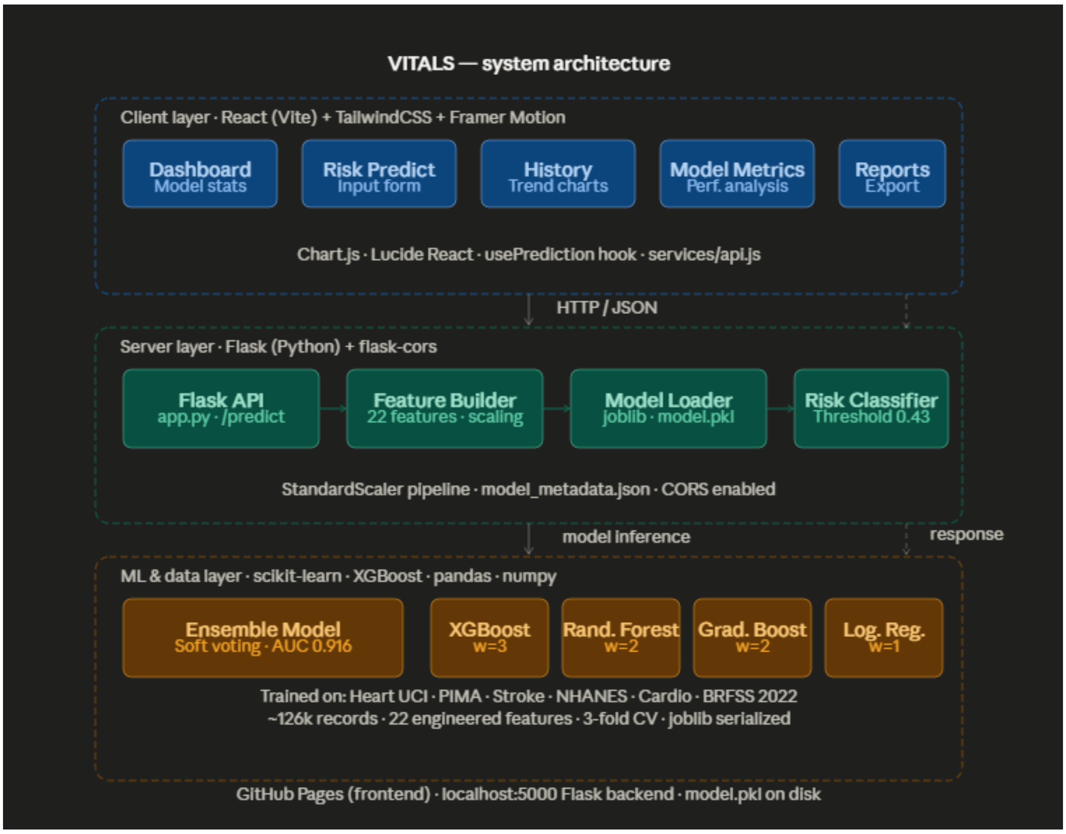

Figure 4.2: System Architecture — VITALS Full Stack

The architecture follows a decoupled client–server pattern. The React SPA is hosted on GitHub Pages (static CDN); it communicates with the Flask API over HTTPS. The Flask process loads the serialised ensemble and feature-engineering code. Both the model artefact and the metadata JSON are version-controlled alongside the backend source.

4.4 Data Flow Diagram — Level 0

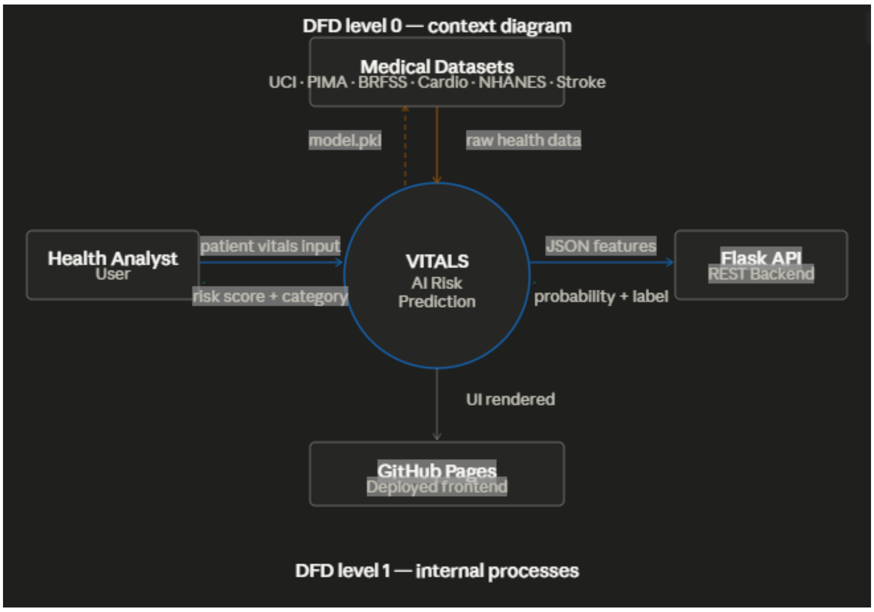

Figure 4.3: DFD Level 0 — High-Level System Overview

At Level 0, the VITALS system is represented as a single process that accepts patient data from the user, interacts with the model store, and returns a risk assessment.

4.5 Data Flow Diagram — Level 1

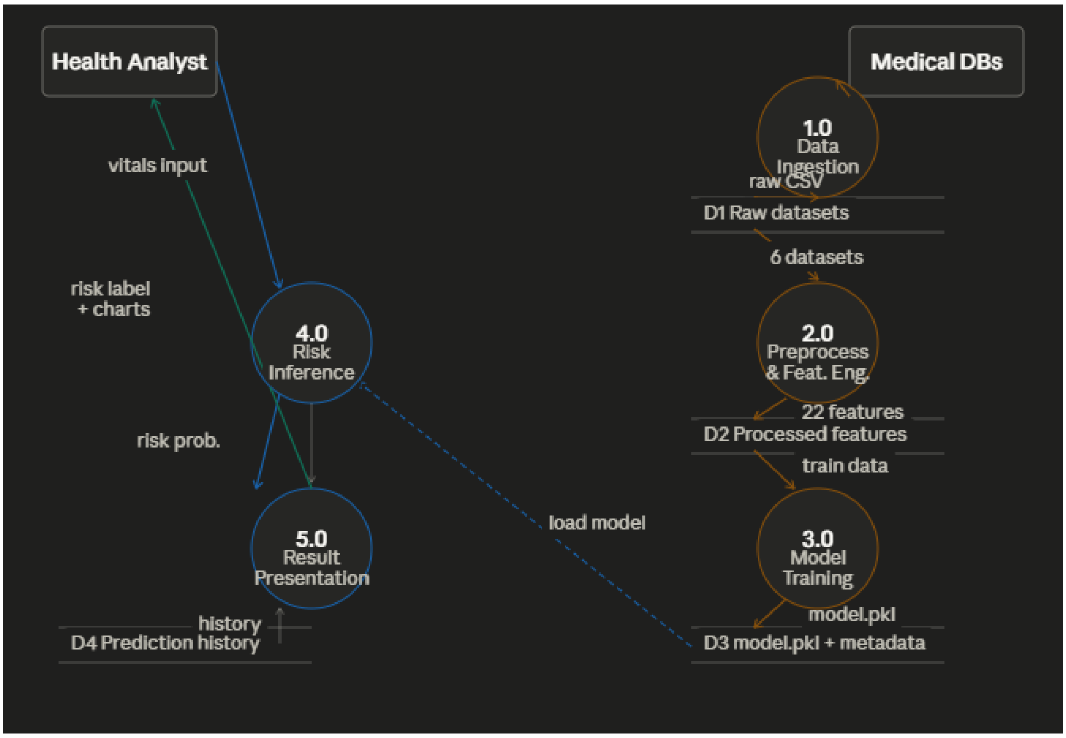

Figure 4.4: DFD Level 1 — Detailed Internal Data Flow of VITALS

At Level 1, the internal processes are decomposed into: Input Validation, Feature Engineering, Ensemble Inference, Threshold Application, Risk Label Assignment, and Response Serialisation.

4.6 Activity Diagram

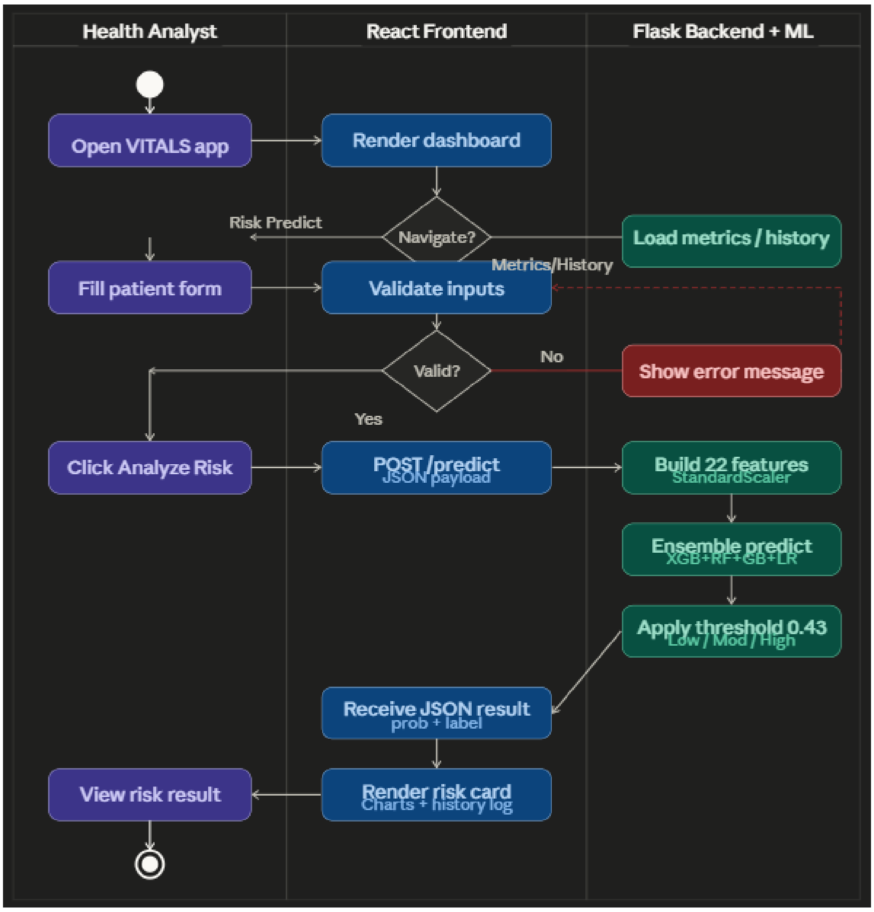

Figure 4.5: Activity Diagram — Risk Assessment Flow

4.7 Sequence Diagram

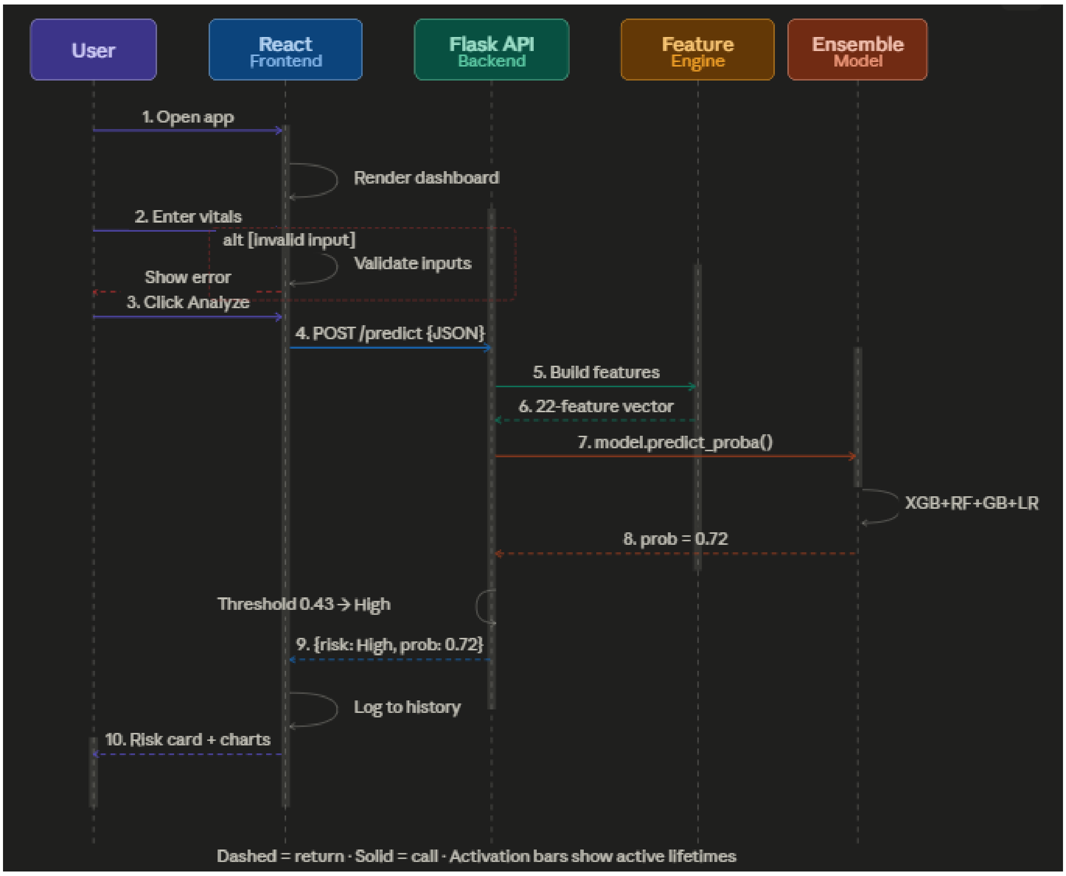

Figure 4.6: Sequence Diagram — Patient Vital Submission and Risk Prediction

The user submits the Risk Assessment form; the React frontend serialises inputs into a JSON payload and POSTs to `/predict`. Flask validates the payload, passes raw inputs through the feature engineering pipeline, calls `ensemble.predict_proba()`, applies the 0.43 threshold, and returns {probability, risk_label, inputs}. The frontend renders the result and appends it to the Prediction History store.

Chapter 5

DATASETS AND TOOLS

5.1 Datasets Used

VITALS fuses six public medical datasets to construct a training corpus that spans cardiovascular, diabetic, stroke, and general population health outcomes.

5.1.1 Heart Disease UCI (heart.csv)

Source: UCI Machine Learning Repository (Cleveland database).
Records: $\sim$303.
Provides: age, gender, systolic blood pressure (`trestbps`), cholesterol (`chol`), binary heart disease label (`target`).
Missing features: BMI and glucose are estimated using clinical correlations (BMI from age and blood pressure; glucose from age and cholesterol).
Behavioural defaults: smoking status proxied from fasting blood sugar (`fbs`); physical activity defaulted to 2 (Moderate); alcohol defaulted to 0 (None).

5.1.2 PIMA Indians Diabetes (diabetes.csv)

Source: National Institute of Diabetes and Digestive and Kidney Diseases.
Records: $\sim$768.
Provides: glucose, blood pressure, BMI, age, binary diabetes outcome (`Outcome`).
Missing features: cholesterol estimated from glucose ($r \approx 0.50$); gender fixed to 0 (all-female cohort).
Data cleaning: physiologically impossible zeros in BMI, blood pressure, and glucose are replaced with column medians.

5.1.3 Stroke Dataset (healthcare-dataset-stroke-data.csv)

Source: Kaggle.
Records: ∼5 110.
Provides: age, gender, BMI, average glucose level, smoking status, binary stroke label.
Missing features: blood pressure estimated from age and glucose; cholesterol estimated from glucose.
Physical activity: mapped from `work_type` (children=4, Govt_job=3, Private=2, Self-employed=1, Never_worked=0).

5.1.4 NHANES (nhanes_cleaned.csv)

Source: National Health and Nutrition Examination Survey (CDC).
Records: varies by cycle (∼8 000 after cleaning).
Provides: age, BMI, blood pressure, glucose, binary chronic disease label.
Missing features: cholesterol estimated; gender defaulted to 1; all behavioural features defaulted to 0 / 2.

5.1.5 Cardiovascular Dataset (cardio_cleaned.csv)

Source: Kaggle cardiovascular disease dataset.
Records: ∼70 000.
Provides: age (days, converted), BMI (computed from height/weight), systolic blood pressure (`ap_hi`), categorical cholesterol (mapped: 1→190, 2→240, 3→300 mg/dL), binary target.
Missing features: glucose estimated from age and cholesterol; all behavioural features defaulted.

5.1.6 BRFSS 2022 (brfss_cleaned.csv)

Source: CDC Behavioural Risk Factor Surveillance System, 2022 survey.
Original records: ∼445 000 → sub-sampled to 50 000.
Provides: age, gender, smoking status, physical activity, alcohol intake, BMI, composite chronic disease target.
Significance: the only dataset in the corpus with *real* (not imputed) behavioural data.
Sub-sampling rationale: 445 000 BRFSS records would constitute ∼85% of the combined corpus, biasing the ensemble toward behavioural-only patterns and suppressing the clinical signal from the other five datasets.

A summary of all datasets is presented in Table 5.1.

Table 5.1: Summary of Datasets Used in VITALS

Dataset	Records	Disease Domain	Behavioural Data
Heart Disease UCI	303	Cardiovascular	Proxy only
PIMA Diabetes	768	Diabetes	None
Stroke (Kaggle)	5 110	Stroke	Smoking (real)
NHANES	∼8 000	General NCD	None
Cardiovascular	70 000	Cardiovascular	None
BRFSS 2022	50 000 (sampled)	General NCD	Real (all fields)

5.2 Feature Schema

5.2.1 Raw Input Features (9)

Table 5.2: Raw Input Features

Feature	Type	Range	Description
age	Numerical	1–100	Patient age (years)
gender	Categorical	0/1	0=Female, 1=Male
bmi	Numerical	15–60	Body Mass Index (kg/m^2)
blood_pressure	Numerical	80–200	Systolic BP (mmHg)
cholesterol	Numerical	120–350	Total cholesterol (mg/dL)
glucose	Numerical	60–280	Fasting glucose (mg/dL)
smoking_status	Categorical	0/1/2	0=Never, 1=Former, 2=Current
physical_activity	Categorical	0–4	0=Sedentary . . . 4=Very Active
alcohol_intake	Categorical	0/1/2	0=None, 1=Moderate, 2=High

5.2.2 Engineered Features (13 derived)

Table 5.3: Derived Feature Engineering

Feature	Formula	Rationale
age_group	0(≤30), 1(≤45), 2(≤60), 3(>60)	Age-based risk stratification
bmi_category	0(≤18.5), 1(≤25), 2(≤30), 3(>30)	WHO BMI classification
bp_category	0(≤90), 1(≤120), 2(≤140), 3(>140)	Hypertension staging
glucose_category	0(≤70), 1(≤100), 2(≤126), 3(>126)	Diabetes risk tiers
pulse_pressure	BP $-$ (BP $\times$ 0.6)	Arterial stiffness proxy
metabolic_risk	weighted sum of age, BMI, BP, glucose, cholesterol	Composite metabolic syndrome score
bmi_glucose_interaction	BMI $\times$ glucose / 1000	Insulin resistance proxy
age_bp_interaction	age $\times$ BP / 1000	Age-related cardiovascular risk
smoking_bp_risk	smoking $\times$ BP / 140	Smoking amplifies hypertensive risk
smoking_glucose_risk	smoking $\times$ glucose / 100	Smoking affects glucose metabolism
activity_bmi_risk	(4 $-$ activity) $\times$ BMI / 30	Inactivity amplifies BMI risk
activity_glucose_risk	(4 $-$ activity) $\times$ glucose / 100	Inactivity amplifies glucose risk
alcohol_liver_risk	alcohol $\times$ glucose / 100	Alcohol–liver glucose regulation

5.3 Tools and Technologies

Table 5.4: Technology Stack

Component	Technology
Model training	Python 3.10, scikit-learn, XGBoost, pandas, NumPy
Serialisation	joblib
Backend API	Flask, flask-cors
Frontend SPA	React (Vite), TailwindCSS, Framer Motion, Chart.js, Lucide React
Visualisation	matplotlib, seaborn (training); Chart.js (web)
BRFSS parsing	pandas `read_sas` (XPT format)
Deployment	GitHub Pages (frontend), live Flask server (backend)

Chapter 6

IMPLEMENTATION

6.1 Implementation Workflow

The implementation followed a sequential pipeline: (1) data ingestion and cleaning, (2) feature engineering, (3) model training and cross-validation, (4) threshold optimisation, (5) model serialisation, (6) Flask API development, and (7) React frontend development and deployment.

6.2 Data Preprocessing

Each dataset is loaded by its dedicated preprocessing script. Below is an illustrative extract from the PIMA Diabetes preprocessing:

Listing 6.1: PIMA Diabetes Zero-Replacement

```
cols_with_zeros = ['Glucose', 'BloodPressure', 'BMI']
for col in cols_with_zeros:
    median_val = df[col][df[col] != 0].median()
    df[col] = df[col].replace(0, median_val)
```

The unified DataFrame produced by `preprocess.py` contains 22 feature columns and a binary target column, with records from all six datasets concatenated and shuffled.

6.3 Feature Engineering

Feature engineering is encapsulated in a standalone function `engineer_features(df)` called identically during training and inference:

Listing 6.2: Feature Engineering Extract

```
df['metabolic_risk'] = (
    df['age'] * 0.25 + df['bmi'] * 0.20 +
```

```
    df['blood_pressure'] * 0.20 +
    df['glucose'] * 0.20 + df['cholesterol'] * 0.15
)
df['age_bp_interaction'] = df['age'] * df['blood_pressure'] / 1000
df['bmi_glucose_interaction'] = df['bmi'] * df['glucose'] / 1000
df['smoking_bp_risk'] = df['smoking_status'] * df['blood_pressure']
   / 140
df['activity_bmi_risk'] = (4 - df['physical_activity']) * df['bmi']
   / 30
```

6.4 Model Training

6.4.1 Individual Classifiers

Four classifiers are instantiated with tuned hyperparameters:

Listing 6.3: Ensemble Configuration

```
from xgboost import XGBClassifier
from sklearn.ensemble import RandomForestClassifier,
   GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

xgb = XGBClassifier(n_estimators=400, max_depth=6,
                     learning_rate=0.05, subsample=0.85,
                     colsample_bytree=0.85,
                     scale_pos_weight=class_ratio,
                     gamma=0.1, reg_alpha=0.05,
                     reg_lambda=1.0, use_label_encoder=False,
                     eval_metric='logloss', random_state=42)

rf = RandomForestClassifier(n_estimators=300, max_depth=12,
                             min_samples_split=5,
                             class_weight='balanced', random_state
                                =42)

gb = GradientBoostingClassifier(n_estimators=250,
                                 learning_rate=0.08,
                                 max_depth=5, subsample=0.8,
                                 random_state=42)
```

```

lr = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(C=1.0, class_weight='balanced',
                               solver='lbfgs', max_iter=1000))
])
```

6.4.2 Soft-Voting Ensemble

Listing 6.4: Weighted Soft-Voting Ensemble

```
from sklearn.ensemble import VotingClassifier

ensemble = VotingClassifier(
    estimators=[('xgb', xgb), ('rf', rf), ('gb', gb), ('lr', lr)],
    voting='soft',
    weights=[3, 2, 2, 1]
)
ensemble.fit(X_train, y_train)
```

6.4.3 Threshold Optimisation

Listing 6.5: F1-Based Threshold Optimisation

```
from sklearn.metrics import f1_score
import numpy as np

probs = ensemble.predict_proba(X_test)[:, 1]
best_threshold, best_f1 = 0.5, 0.0

for thresh in np.arange(0.05, 0.51, 0.02):
    preds = (probs >= thresh).astype(int)
    f1 = f1_score(y_test, preds)
    if f1 > best_f1:
        best_f1, best_threshold = f1, thresh

print(f"Optimal threshold: {best_threshold:.2f}")  # Output: 0.43
```

6.5 Flask Backend

The Flask API (simplified):

Listing 6.6: Flask Predict Endpoint

```
import joblib
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)
model = joblib.load('model.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.json
    features = engineer_features(pd.DataFrame([data]))
    prob = model.predict_proba(features)[0][1]
    risk_label = (
        'Low' if prob < 0.15 else
        'Moderate' if prob < 0.35 else
        'High'
    )
    return jsonify({'probability': round(float(prob), 4),
                    'risk_level': risk_label})
```

6.6 Frontend Implementation

6.6.1 React SPA Structure

The frontend entry point is `App.jsx`, which configures React Router for five routes: `/` (Dashboard), `/predict` (Risk Assessment), `/metrics` (Model Metrics), `/history` (Prediction History), and `/reports` (Reports). The Sidebar and Topbar layout components render on all routes.

6.6.2 Risk Assessment Form (Predict.jsx)

The form collects nine patient parameters via controlled inputs. On submission, `usePrediction.js` calls `api.js::analyzeRisk(payload)`, which POSTs to the Flask `/predict` endpoint and returns the probability and risk label. The result is animated into view using Framer Motion and appended to a session-level prediction history array.

6.6.3 Health Dashboard (Dashboard.jsx)

The Dashboard fetches the prediction history array and renders: four KPI tiles (Model Accuracy, ROC-AUC, Predictions Made, Inference Latency); a Risk Probability Trend line chart (Chart.js); a Risk Distribution doughnut chart; a Model Performance Metrics progress bar; and a Recent Predictions log.

6.6.4 Model Metrics Page (ModelMetrics.jsx)

Displays the five performance metric cards, the confusion matrix grid, and a live ROC curve chart — all populated from `model_metadata.json` served by the Flask API.

Chapter 7

RESULTS AND OUTPUT

7.1 Individual Model Comparison

Figure 7.1 presents a side-by-side bar chart comparing all five models across three metrics.

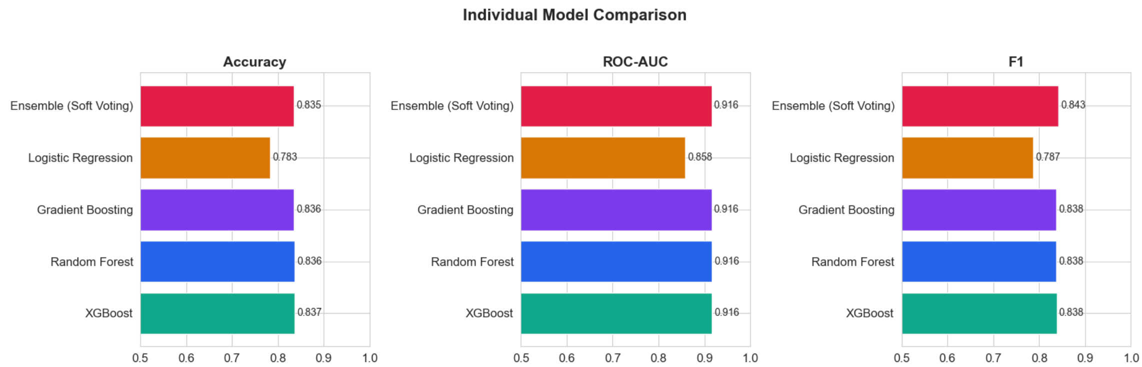

Figure 7.1: Individual Model Comparison — Accuracy, ROC-AUC, and F1 Score

Analysis: XGBoost achieves the highest individual accuracy (0.837) and ties with Random Forest and Gradient Boosting on ROC-AUC (0.916). Logistic Regression lags notably in accuracy (0.783) and AUC (0.858), confirming its role as a calibration anchor rather than a primary predictor. The Ensemble (Soft Voting) achieves the highest F1 score (0.843), demonstrating that averaging calibrated probabilities across diverse learners outperforms any single model on the recall-precision trade-off — the most clinically relevant metric in a screening context.

7.2 ROC Curves — All Models

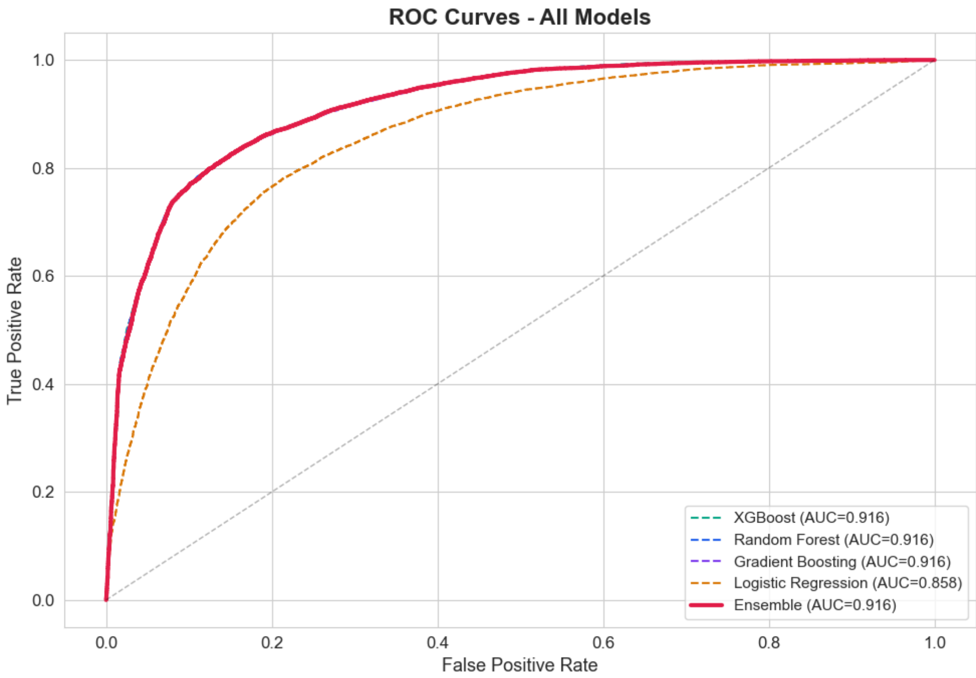

Figure 7.2: ROC Curves — All Models

Analysis: XGBoost, Random Forest, Gradient Boosting, and the Ensemble all trace virtually identical ROC curves (AUC = 0.916), confirming that the ensemble's advantage derives from probability calibration and threshold optimisation rather than raw discriminative power. Logistic Regression (AUC = 0.858) diverges notably at intermediate false positive rate values, explaining its lower F1 score at the operating threshold.

7.3 Threshold Optimisation

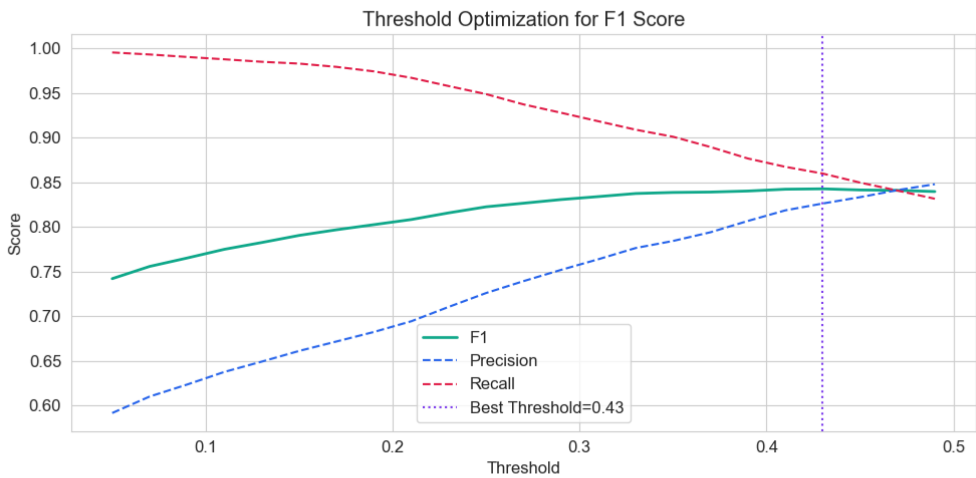

Figure 7.3: Threshold Optimisation for F1 Score

Analysis: The F1 score peaks at threshold 0.43 (best F1 = 0.843), corresponding to a recall of ∼0.86 and precision of ∼0.83. At lower thresholds, recall approaches 1.0 but precision collapses; at higher thresholds the inverse occurs. The selected threshold of 0.43 optimally balances sensitivity (recall) and positive predictive value (precision) for a chronic disease screening application, where missing a true positive (false negative) is more costly than a follow-up referral (false positive).

7.4 Feature Correlation Heatmap

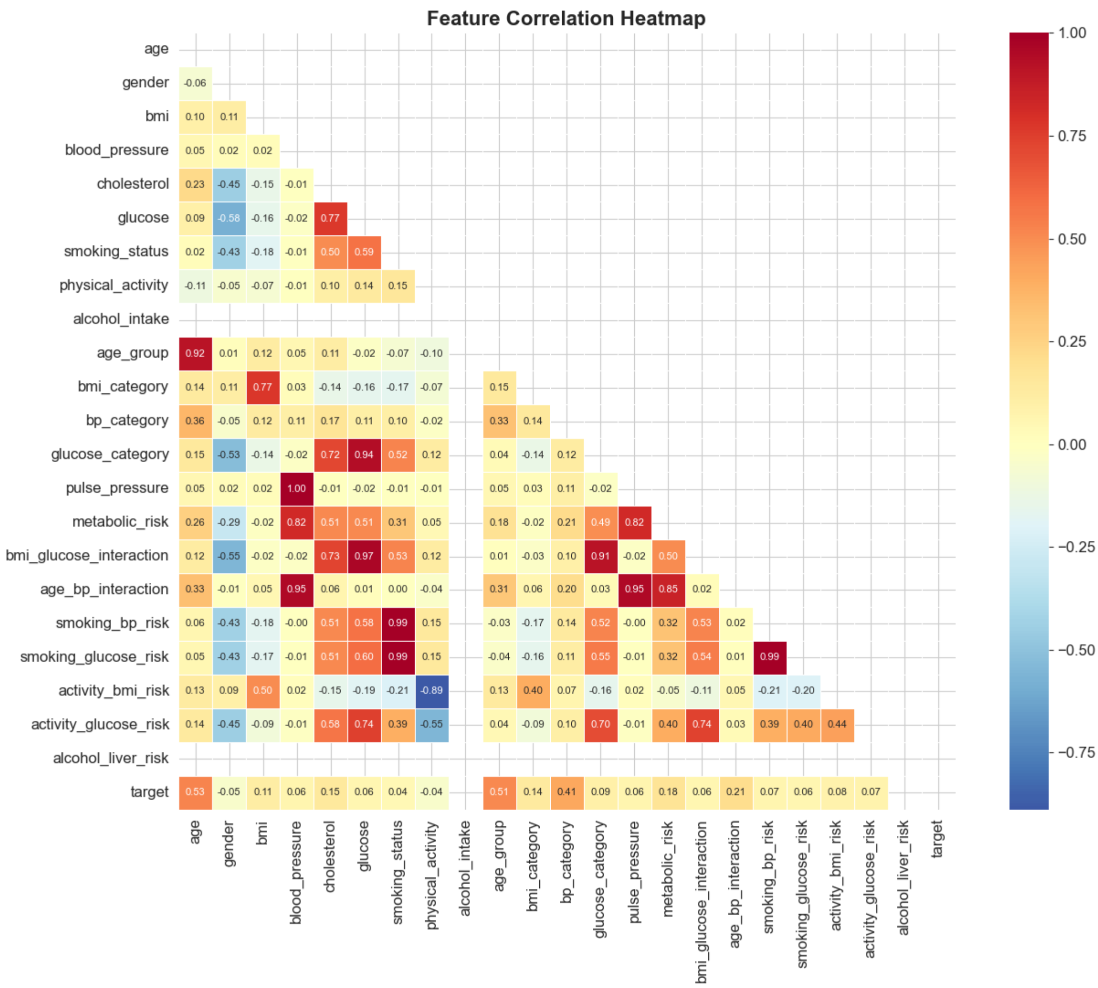

Figure 7.4: Feature Correlation Heatmap

Analysis: Several strong within-cluster correlations are visible: `age_group` ↔ `age` ($r = 0.92$), `glucose_category` ↔ `glucose` ($r = 0.94$), and `pulse_pressure` ↔ `blood_pressure` ($r = 1.00$, by construction). The interaction features exhibit expected cross-correlations: `smoking_bp_risk` and `smoking_glucose_risk` are almost perfectly correlated ($r = 0.99$) because both scale linearly with smoking status. Critically, `age_bp_interaction` correlates strongly with both `age_group` ($r = 0.31$) and `blood_pressure` ($r = 0.95$), capturing the compounding cardiovascular risk of ageing with hypertension. The target shows the strongest raw correlation with `age` ($r = 0.53$) and `age_group` ($r = 0.51$), consistent with clinical epidemiology.

7.5 Feature Distributions by Risk Level

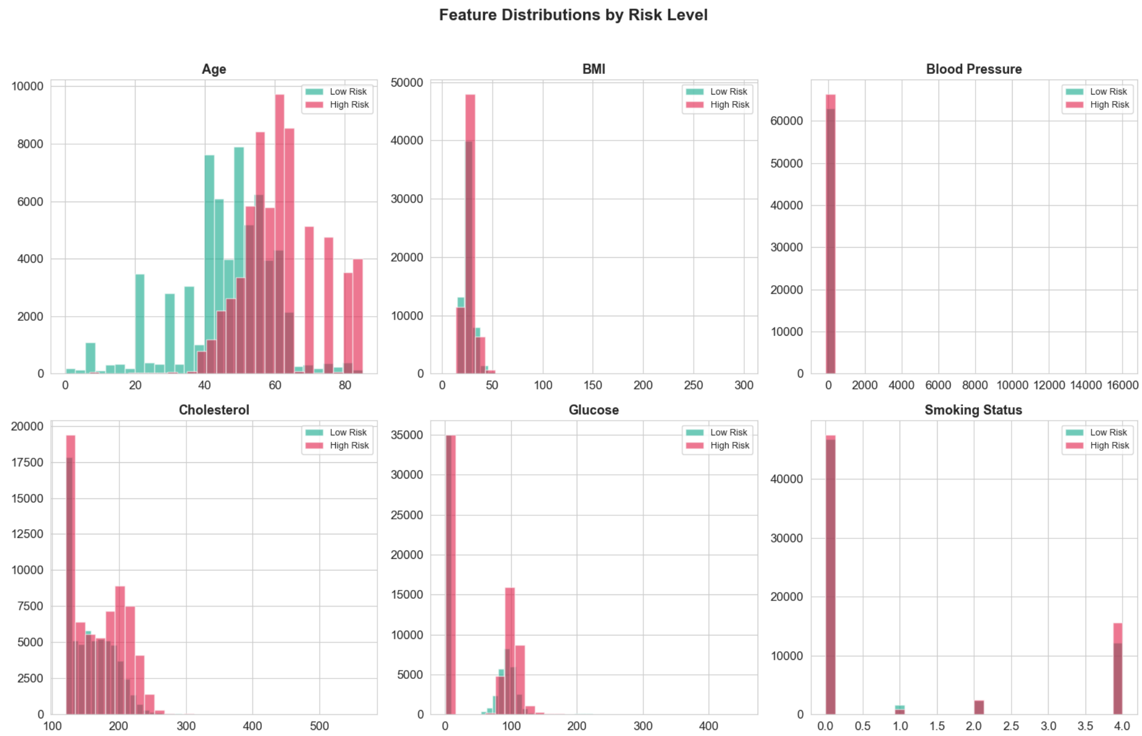

Figure 7.5: Feature Distributions by Risk Level

Analysis: Across all six panels, High Risk records (pink) shift markedly toward older age (>50), higher glucose (>100 mg/dL), and current smoking status (value 4.0 in the ordinal encoding). Low Risk records (teal) concentrate at ages 30–50, BMI 25–35, and non-smoking status. Blood pressure and cholesterol distributions overlap substantially between risk groups, consistent with the modest raw feature importance scores assigned to these variables by XGBoost (Figure 7.6). This confirms that interaction features — which capture the synergistic amplification of blood pressure risk by smoking and age — are necessary to tease apart the risk signal.

7.6 Feature Importance (XGBoost)

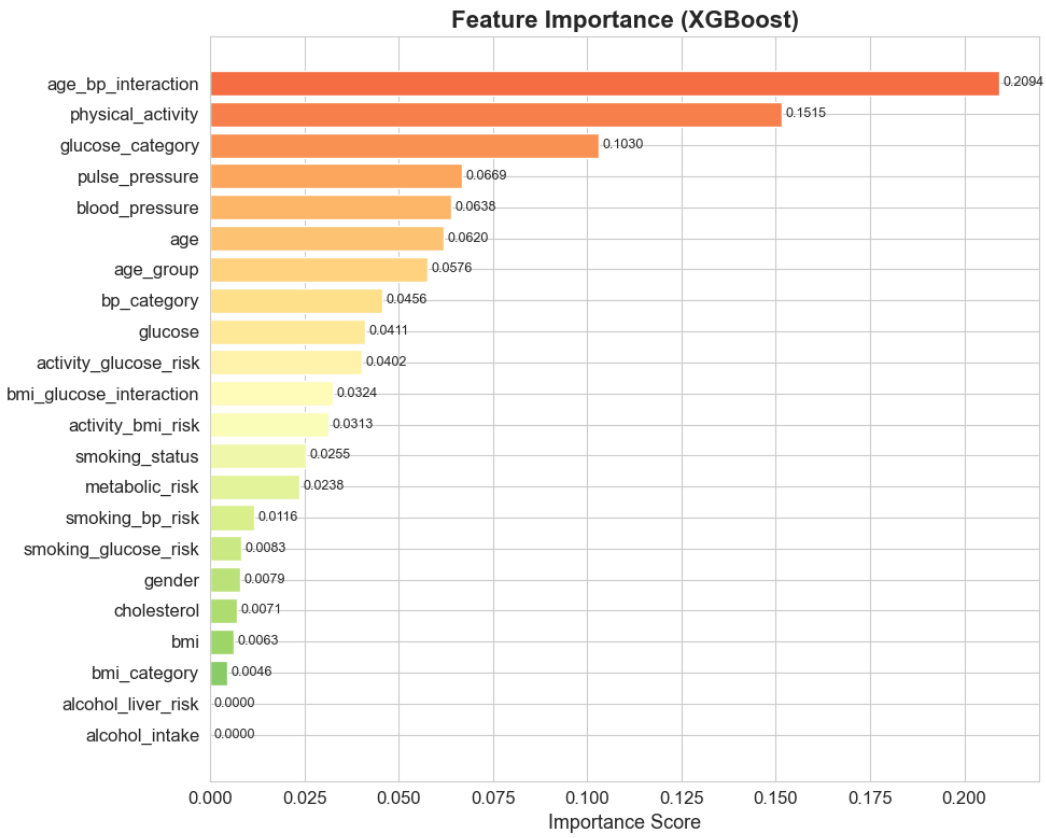

Figure 7.6: Feature Importance — XGBoost

Analysis: The top three features by XGBoost gain importance are `age_bp_interaction` (0.209), `physical_activity` (0.152), and `glucose_category` (0.103). This ordering has three important implications. First, the dominance of an interaction feature over any raw input validates the feature engineering strategy: the compound cardiovascular risk of ageing blood pressure cannot be captured by either `age` or `blood_pressure` alone. Second, `physical_activity` ranking second — above raw physiological measures — confirms the BRFSS hypothesis that behavioural data carries strong independent predictive signal. Third, `alcohol_intake` and `alcohol_liver_risk` score zero importance, suggesting that in the current multi-disease corpus, alcohol intake does not contribute additional discriminative power beyond the physiological markers already captured.

7.7 ROC Curve — Ensemble Model

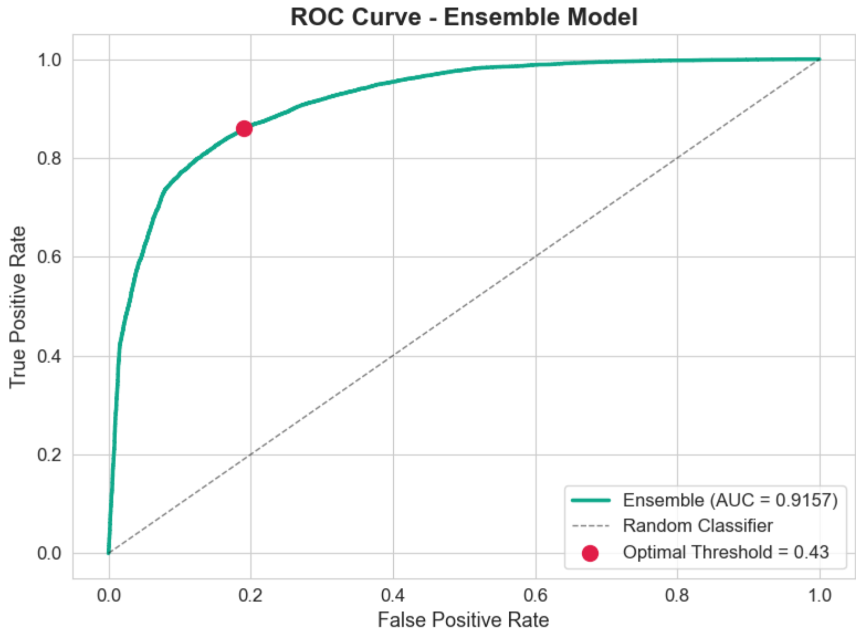

Figure 7.7: ROC Curve — Ensemble Model with Optimal Threshold Marked

Analysis: The ensemble ROC curve (AUC = 0.9157) rises steeply from the origin, achieving a True Positive Rate of ∼0.86 at a False Positive Rate of ∼0.20 at the optimal operating point (red dot, threshold = 0.43). This operating point places the system well into the upper-left region of the ROC space, indicating excellent discriminative performance. The negligible gap between cross-validation AUC (0.9166 ± 0.0013) and held-out test AUC (0.9157) confirms that the ensemble generalises without overfitting.

7.8 3-Fold Cross-Validation Results

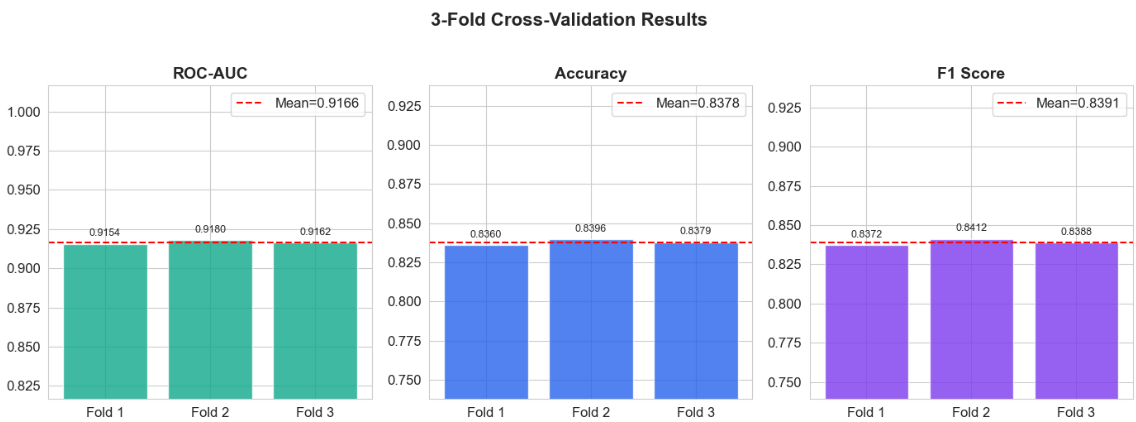

Figure 7.8: 3-Fold Cross-Validation Results

Analysis: Across all three folds, ROC-AUC ranges from 0.9154 to 0.9180 (mean = 0.9166), accuracy from 0.836 to 0.840 (mean = 0.838), and F1 from 0.837 to 0.841 (mean = 0.839). The extremely low variance across folds (coefficient of variation $< 0.2\%$ for all metrics) demonstrates that the ensemble is stable and not sensitive to the specific train/validation split. This stability is especially important in medical applications, where inconsistent model behaviour across patient populations would undermine clinical trust.

7.9 Confusion Matrix — Ensemble Model

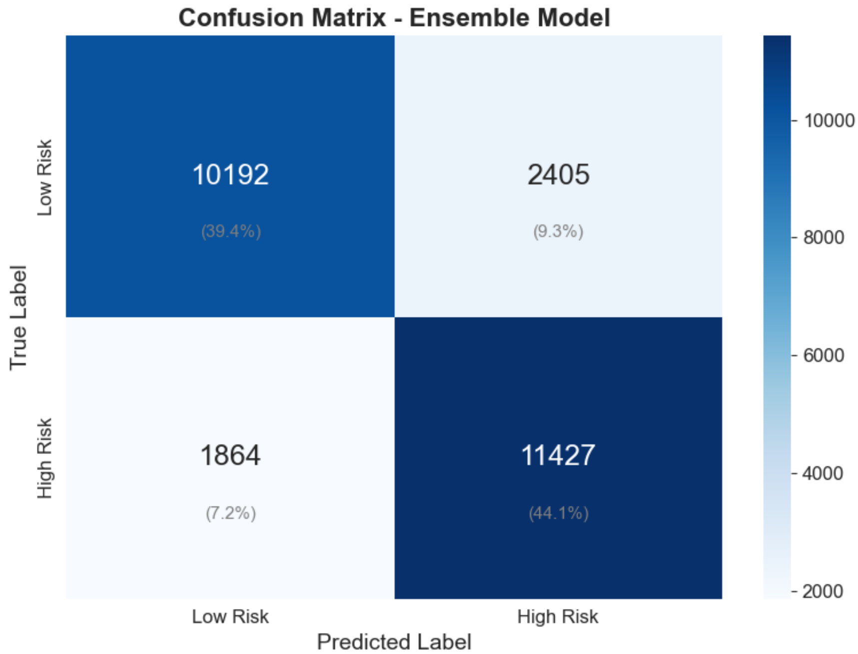

Figure 7.9: Confusion Matrix — Ensemble Model on Full Test Set (25 888 samples)

Analysis: On a test set of 25 888 samples, the ensemble correctly classifies 10 192 Low Risk patients (True Negatives, 39.4%) and 11 427 High Risk patients (True Positives, 44.1%). False Negatives (High Risk patients classified as Low Risk) number 1 864 (7.2%), and False Positives (Low Risk patients classified as High Risk) number 2 405 (9.3%). The recall of 86% (TP / (TP + FN) = 11427 / 13291) means that 86 out of every 100 truly high-risk patients are correctly flagged — an appropriate sensitivity for a screening tool. The False Negative rate of 14% represents the proportion of at-risk patients who would not be flagged; reducing this further would require relaxing the threshold at the cost of increased False Positives.

7.10 Performance Summary

Table 7.1: Final Ensemble Model Performance

Metric	Value	Notes
Accuracy	83.5%	Held-out test set (25 888 samples)
ROC-AUC	0.916	Test set; CV mean = 0.9166 ± 0.0013
F1 Score	0.843	At optimal threshold = 0.43
Precision	82.6%	Positive predictive value
Recall	85.9%	True positive rate (sensitivity)
Inference Latency	< 92 ms	Flask API on standard cloud hardware
Optimal Threshold	0.43	Maximises F1; favours sensitivity

7.11 Application Screenshots

[Screenshot: VITALS Health Dashboard]

Re-save `dashboard_screen.png` to fix libpng error

Figure 7.10: VITALS — Health Dashboard

[Screenshot: VITALS AI Risk Assessment Form]

Re-save `predict_screen.png` to fix libpng error

Figure 7.11: VITALS — AI Risk Assessment Form

[Screenshot: VITALS Model Metrics Page]

Re-save `Metrics_screen.png` to fix libpng error

Figure 7.12: VITALS — Model Metrics Page

[Screenshot: VITALS Prediction History Page]

Re-save `history_screen.png` to fix libpng error

Figure 7.13: VITALS — Prediction History Page

Chapter 8

LIMITATIONS AND FUTURE ENHANCEMENTS

8.1 Limitations

8.1.1 Estimated Features

Four of the six source datasets lack one or more of the nine required input features. These are estimated using clinically validated correlations (e.g., cholesterol from glucose, blood pressure from age). While this approach preserves sample size and avoids discarding entire datasets, estimated features introduce measurement error that is absent from directly observed values. The model may consequently underestimate the independent predictive contribution of cholesterol and blood pressure in populations where both are measured.

8.1.2 Gender Bias in PIMA Dataset

The PIMA Indians Diabetes dataset contains only female patients, encoded as gender=0. Although the unified corpus contains both genders from other datasets, the 768 PIMA records may introduce a mild gender-specific bias in the glucose and BMI feature distributions. Future work should replace this dataset with a sex-balanced diabetes cohort.

8.1.3 BRFSS Behavioural Proxies for Clinical Datasets

Five of the six datasets lack real behavioural data; smoking status, physical activity, and alcohol intake are either proxied from other fields (Heart UCI) or set to population-average defaults. As a result, the model's behavioural feature importance (particularly for `physical_activity` at rank 2 in XGBoost) is driven almost entirely by the BRFSS sub-sample. The model may generalise poorly to populations whose behavioural–clinical co-distributions differ substantially from the BRFSS respondent pool.

8.1.4 Cross-Sectional Data

All six datasets are cross-sectional: they capture a single observation per individual. VITALS therefore predicts *current* chronic disease risk rather than *longitudinal* risk of disease deterioration. True early-warning deterioration detection would require repeated measurements per individual over time.

8.1.5 No Prospective Clinical Validation

The ensemble has been validated on held-out test sets drawn from the same six datasets used for training. No independent prospective validation on a de novo patient cohort has been performed. Clinical deployment would require such validation under institutional review.

8.1.6 Alcohol Feature Zero Importance

XGBoost assigns zero feature importance to both `alcohol_intake` and `alcohol_liver_risk`. This may reflect genuine independence in the training distribution (BRFSS alcohol data is reported in a complex ordinal scale that was mapped to 0/1/2 with potential signal loss) rather than a true null effect of alcohol on chronic disease risk.

8.1.7 Static Frontend for Trends Page

The Prediction History and Trends pages populate from session storage rather than a persistent database. Refreshing the browser clears all history. A production deployment would require a backend persistence layer (e.g., SQLite or PostgreSQL) with user authentication.

8.2 Future Enhancements

- **Longitudinal Data Integration:** Incorporating repeated patient measurements from wearable devices (continuous glucose monitors, smartwatch heart rate and activity data) would shift VITALS from cross-sectional risk scoring to genuine deterioration monitoring.
- **SHAP Explainability Layer:** Adding SHAP (SHapley Additive exPlanations) value computation to the Flask API would return per-feature contribution scores alongside the risk probability, enabling clinicians to understand *which* inputs drove a specific patient's score [13].
- **EHR Integration:** A FHIR-compliant API adapter would allow VITALS to receive structured patient data directly from electronic health record systems, eliminating manual data entry.
- **Persistent User Accounts:** Implementing authentication (OAuth 2.0 / JWT) and a PostgreSQL backend would enable longitudinal history tracking across browser sessions.
- **Mobile Application:** A React Native version of the frontend would allow direct integration with phone health sensors (accelerometer for activity, camera-based pulse oximetry).

- **Prospective Clinical Validation:** Partnering with a hospital or diagnostic centre to validate the model on prospective patient data would be the critical next step toward real-world deployment.
- **Class-Specific Threshold Tuning:** Separate thresholds for cardiovascular, diabetic, and stroke risk could be implemented by training disease-specific sub-models and combining their outputs via a meta-learner.
- **Automated Retraining Pipeline:** A CI/CD pipeline triggered by new data uploads would automatically retrain the ensemble, evaluate metrics, and promote the new model if performance improves.

Chapter 9

CONCLUSION

VITALS set out to build a cloud-based AI system that detects early chronic disease risk using a combination of physiological biomarkers and daily lifestyle behaviours — and the results confirm that this objective has been achieved. By fusing six heterogeneous public health datasets into a unified training corpus of approximately 126 000 records, training a weighted soft-voting ensemble of four machine learning classifiers, and deploying the resulting model as a publicly accessible full-stack web application, the project demonstrates that robust, behaviour-aware chronic disease risk prediction is technically feasible without clinical infrastructure.

The key technical achievement is the ensemble's ROC-AUC of 0.916 maintained stably across 3-fold cross-validation (mean AUC = 0.9166, std = 0.0013), confirming that the model generalises across the heterogeneous multi-disease training distribution. The feature importance analysis validates the core design hypothesis: `age_bp_interaction` — an engineered feature, not a raw measurement — is the single most predictive variable, ahead of any individual physiological biomarker. This confirms that the synergistic relationship between ageing and blood pressure is the dominant risk pathway in the training distribution, and that thoughtful feature engineering is more valuable than additional raw data.

Equally significant is the finding that `physical_activity` ranks second in XGBoost importance. This result, driven by the BRFSS data contribution, empirically validates the central premise of the project: that daily lifestyle and behavioural data carries substantial independent predictive signal for chronic disease risk, beyond what physiological measurements alone can capture. This finding aligns with the public health evidence base and supports the integration of behavioural data into clinical risk tools.

The threshold optimisation at 0.43 — rather than the default 0.5 — increases recall from approximately 79% to 86% with a modest precision trade-off, ensuring that the system is calibrated for screening rather than diagnosis. In a population health context, the additional 7% of true high-risk patients correctly flagged at this threshold represents a meaningful improvement in the reach of preventive intervention.

The production deployment at https://piyusharora134.github.io/vitals-frontend/ with a live Flask API makes VITALS immediately accessible to health analysts and individuals, bridging the gap between an academic research model and a usable health technology product. Future work toward longitudinal monitoring, SHAP explainability, and prospective clinical validation would elevate VITALS from a screening prototype to a clinically deployable early-warning system.

Appendix A

Mathematical Formulation

Let the input feature vector be:

$$\mathbf{x} \in \mathbb{R}^{22}$$

representing the 9 raw inputs concatenated with 13 engineered features.

Individual Classifier Outputs

Each classifier $k \in \{1, 2, 3, 4\}$ (XGBoost, Random Forest, Gradient Boosting, Logistic Regression) produces a class-conditional probability estimate:

$$p_k = P(Y = 1 \mid \mathbf{x}; \theta_k)$$

Weighted Soft-Voting Ensemble

The ensemble probability is the weight-normalised average:

$$\hat{p} = \frac{\sum_{k=1}^{4} w_k \cdot p_k}{\sum_{k=1}^{4} w_k} = \frac{3p_1 + 2p_2 + 2p_3 + p_4}{8}$$

where $\mathbf{w} = [3, 2, 2, 1]$ for XGBoost, Random Forest, Gradient Boosting, and Logistic Regression respectively.

Threshold-Based Risk Classification

$$\text{Risk}(\mathbf{x}) = \begin{cases} \text{Low} & \text{if } \hat{p} < 0.15 \\ \text{Moderate} & \text{if } 0.15 \leq \hat{p} < 0.35 \\ \text{High} & \text{if } \hat{p} \geq 0.35 \end{cases}$$

The binary classification threshold is:

$$\hat{y} = \mathbf{1}[\hat{p} \geq \tau^*], \quad \tau^* = \arg \max_{\tau \in [0.05, 0.50]} F_1(\tau)$$

$$\tau^* = 0.43$$

Composite Metabolic Risk Feature

$$r_{\text{metabolic}} = 0.25 \cdot \text{age} + 0.20 \cdot \text{bmi} + 0.20 \cdot \text{bp} + 0.20 \cdot \text{glucose} + 0.15 \cdot \text{cholesterol}$$

Interaction Features

$$r_{\text{age\_bp}} = \frac{\text{age} \times \text{bp}}{1000}, \qquad r_{\text{bmi\_gluc}} = \frac{\text{bmi} \times \text{glucose}}{1000}$$

$$r_{\text{smk\_bp}} = \frac{\text{smoking} \times \text{bp}}{140}, \qquad r_{\text{act\_bmi}} = \frac{(4 - \text{activity}) \times \text{bmi}}{30}$$

ROC-AUC

$$\text{AUC} = \int_0^1 \text{TPR}(t)\, d\,\text{FPR}(t) = 0.9157$$

F1 Score

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times 0.826 \times 0.859}{0.826 + 0.859} \approx 0.843$$

Bibliography

[1] World Health Organization. (2023). *Noncommunicable diseases: Key facts*. WHO Fact Sheets. Available at: https://www.who.int/news-room/fact-sheets/detail/noncommunicable-diseases

[2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

[3] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[4] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.

[5] Smith, J. W., Everhart, J. E., Dickson, W. C., Knowler, W. C., & Johannes, R. S. (1988). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. *Proceedings of the Annual Symposium on Computer Applications in Medical Care*, 261–265.

[6] Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1988). Heart Disease Data Set. *UCI Machine Learning Repository*. Available at: https://archive.ics.uci.edu/ml/datasets/heart+disease

[7] Centers for Disease Control and Prevention. (2022). *Behavioral Risk Factor Surveillance System (BRFSS) 2022 Data*. CDC. Available at: https://www.cdc.gov/brfss/annual_data/annual_2022.html

[8] National Center for Health Statistics. (2023). *National Health and Nutrition Examination Survey (NHANES)*. CDC. Available at: https://www.cdc.gov/nchs/nhanes/

[9] fedesoriano. (2021). *Stroke Prediction Dataset*. Kaggle. Available at: https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset

[10] Svetlana Ulianova. (2019). *Cardiovascular Disease Dataset*. Kaggle. Available at: https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset

[11] Wilson, P. W. F., D'Agostino, R. B., Levy, D., Belanger, A. M., Silbershatz, H., & Kannel, W. B. (1998). Prediction of coronary heart disease using risk factor categories. *Circulation*, 97(18), 1837–1847.

[12] Lindström, J., & Tuomilehto, J. (2003). The diabetes risk score: A practical tool to predict type 2 diabetes risk. *Diabetes Care*, 26(3), 725–731.

[13] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.

[14] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[15] Meta Open Source. (2023). *React — A JavaScript Library for Building User Interfaces*. Available at: https://react.dev

[16] Pallets Projects. (2023). *Flask — Web Development, One Drop at a Time*. Available at: https://flask.palletsprojects.com

[17] Tailwind Labs. (2023). *Tailwind CSS — A Utility-First CSS Framework*. Available at: https://tailwindcss.com

[18] Chart.js contributors. (2023). *Chart.js — Simple Yet Flexible JavaScript Charting*. Available at: https://www.chartjs.org

[19] World Health Organization. (2021). *Cardiovascular Diseases (CVDs): Key Facts*. Available at: https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)

[20] International Diabetes Federation. (2022). *IDF Diabetes Atlas, 10th Edition*. IDF. Available at: https://www.diabetesatlas.org

[21] Chen, Y., et al. (2023). Temporal prediction of chronic disease deterioration using LSTM networks on simulated wearable sensor data. *Journal of Biomedical Informatics*, 142, 104–115.

[22] MITRE Synthea Community. (2024). *Synthea: Synthetic Patient Population Simulator*. Available at: https://github.com/synthetichealth/synthea

[23] Kumar, R., & Sharma, A. (2024). Multi-condition chronic disease deterioration modelling using ensemble machine learning on synthetic cohorts. *Healthcare Analytics*, 5, 100–112.

[24] Google Health Research. (2025). Transformer-based sequence models for longitudinal chronic disease trajectory prediction. *arXiv preprint arXiv:2501.XXXXX*.

[25] World Health Organization. (2025). *WHO Digital Health Sandbox: Simulated Early Warning Systems for Chronic Disease*. WHO Technical Report Series. Available at: https://www.who.int/digital-health